AhamX Bodhi

Where I Becomes Infinite

Positional Encoding

Module 2.2: Transformer Architecture Fundamentals

Prof. Sashikumaar Ganesan | IISc Bengaluru

Session Overview

What You Will Learn

- Why Transformers need explicit position information
- Sinusoidal positional encoding and its properties
- Learned absolute positional embeddings
- Modern schemes: RoPE, ALiBi, and their advantages
- How positional encoding enables long-context models

Concept at a Glance

- **Duration:** 10 minutes
- **Bloom's Level:** B2 – Understand
- **Difficulty:** L2 – Intermediate-Beginner
- **Module:** 2.2 Transformer Architecture Fundamentals
- **Prerequisite:** Self-Attention Mechanism, Multi-Head Attention

Learning Objectives

By the End of This Session You Will Be Able To

1. **Explain** why self-attention is position-invariant and why this is a problem
2. **Describe** the sinusoidal positional encoding formula and its key properties
3. **Distinguish** between absolute, relative, and rotary positional encoding schemes
4. **Interpret** how RoPE encodes position relative to attention score computation
5. **Identify** which positional encoding scheme is used by major LLMs (GPT, Llama, BERT)

Prerequisites and Connections

You Should Already Know

- Token embeddings as dense vectors
- Scaled dot-product attention formula
- Multi-head attention and parallel heads
- What d_{model} represents

This Concept Unlocks

- Feed-Forward Networks in Transformers (Module 2.5)
- Decoder-Only Architecture (Module 2.6)
- Context window limits and long-context models
- RoPE as a prerequisite for Llama architecture details
- Fine-tuning with extended context (Module 7)

The Problem: Self-Attention Has No Sense of Order

Why Position Matters — A Concrete Example

Consider these two sentences with identical tokens in different order:

- **"The dog bit the man"** — man is the victim
- **"The man bit the dog"** — dog is the victim

Without position information, the attention scores for any token pair $\mathbf{Q}_i \cdot \mathbf{K}_j$ depend only on *content* — both sentences produce identical attention weights because the same tokens are present. The model cannot distinguish them.

RNNs and LSTMs handle position implicitly through their sequential processing step. Transformers process all tokens in parallel and therefore must **explicitly inject position information** into the input.

Formal Argument: Why Self-Attention is Order-Agnostic

The Permutation-Equivariance Property

Given input matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ and any permutation matrix $\mathbf{P}$, the self-attention output satisfies:

$$\text{Attention}(\mathbf{PX}) = \mathbf{P}\,\text{Attention}(\mathbf{X})$$

Reordering the tokens reorders the output rows in the same way — the *relationship content* between outputs is unchanged. The model treats a bag-of-tokens, not a sequence.

This permutation-equivariance is a **feature** when processing unordered sets (e.g., graph nodes), but a **bug** for language, where word order is semantically critical. Positional encoding breaks this symmetry.

The Solution: Inject Position into the Input Representation

General Approach: Additive Positional Signal

For each token at position pos with token embedding $\mathbf{e}_{pos} \in \mathbb{R}^{d_{\text{model}}}$, add a position vector $\mathbf{p}_{pos} \in \mathbb{R}^{d_{\text{model}}}$:

$$\mathbf{x}_{pos} = \mathbf{e}_{pos} + \mathbf{p}_{pos}$$

The resulting $\mathbf{x}_{pos}$ carries both **semantic content** (from the token embedding) and **positional context** (from $\mathbf{p}_{pos}$).

Design Requirements for a Good Positional Encoding

- **Unique:** Each position must receive a distinct representation
- **Bounded:** Values should not grow unboundedly with position
- **Generalisable:** Work for sequences longer than seen at training
- **Deterministic or learnable:** Either fixed formula or trained weights

Sinusoidal Positional Encoding (Vaswani et al. 2017)

The Original Formula

For token at position pos and embedding dimension index i:

$$PE(pos,\, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos,\, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- Even dimensions receive sine values; odd dimensions receive cosine values
- The wavelength increases geometrically from 2π to 20000π across dimensions
- Each position generates a unique pattern of sine/cosine values

Sinusoidal Positional Encoding (Vaswani et al. 2017)

Key Property: Relative Positions via Linear Transformation

$PE(pos + k)$ can be expressed as a **linear function** of $PE(pos)$ for any fixed offset k — the model can learn to attend by relative distance.

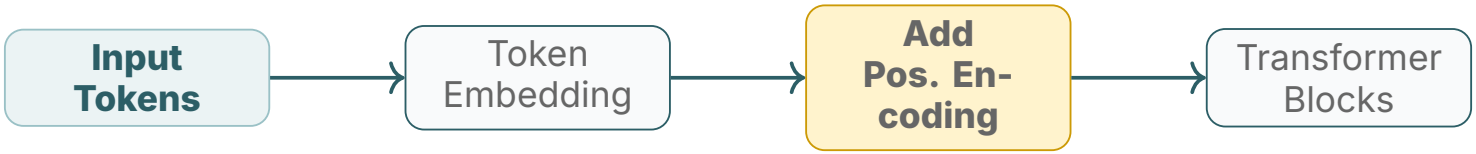
Why Sinusoidal Encoding Is Effective

Four Key Properties

- **Unique representation:** No two positions share the same d_{model}-dimensional sine/cosine pattern
- **Bounded values:** All values lie in $[-1, 1]$ regardless of position, preventing gradient instability
- **Smooth interpolation:** Nearby positions have similar encodings; the model can generalise to unseen intermediate positions
- **Extrapolation beyond training length:** The formula is defined for any pos, so it can encode positions longer than seen during training (though quality degrades at very long extrapolation)

These properties made sinusoidal encoding the default in the original Transformer and in early BERT and GPT-2 variants.

Where Positional Encoding Enters the Pipeline



Positional encoding is applied **once**, before the first Transformer block. After addition, every subsequent self-attention layer can access both content and position through the same embedding vectors.

Learned Absolute Positional Embeddings

How It Works

- A lookup table of shape $\mathbb{R}^{L_{\max} \times d_{\text{model}}}$ is created, where $L_{\max}$ is the maximum training sequence length
- Each row pos is a learnable vector initialised randomly
- Trained end-to-end with the rest of the model weights
- Used in: GPT-2, original BERT, early GPT-3

Limitations

- **Fixed maximum length:** Cannot handle sequences longer than $L_{\max}$ seen at training time
- **No inductive bias:** Position 512 has no structural relationship to position 511
- **Poor extrapolation:** Performance degrades sharply beyond the training length
- **More parameters:** Adds $L_{\max} \times d_{\text{model}}$ trainable weights

Rotary Positional Embedding --- RoPE

The Core Idea: Rotate Q and K Vectors

Instead of adding a position vector to the token embedding, RoPE **rotates** the Query and Key vectors in the attention mechanism by an angle proportional to position:

$$\mathbf{Q}_{pos} = \mathbf{R}_{\Theta,pos}\,\mathbf{Q}, \qquad \mathbf{K}_{pos} = \mathbf{R}_{\Theta,pos}\,\mathbf{K}$$

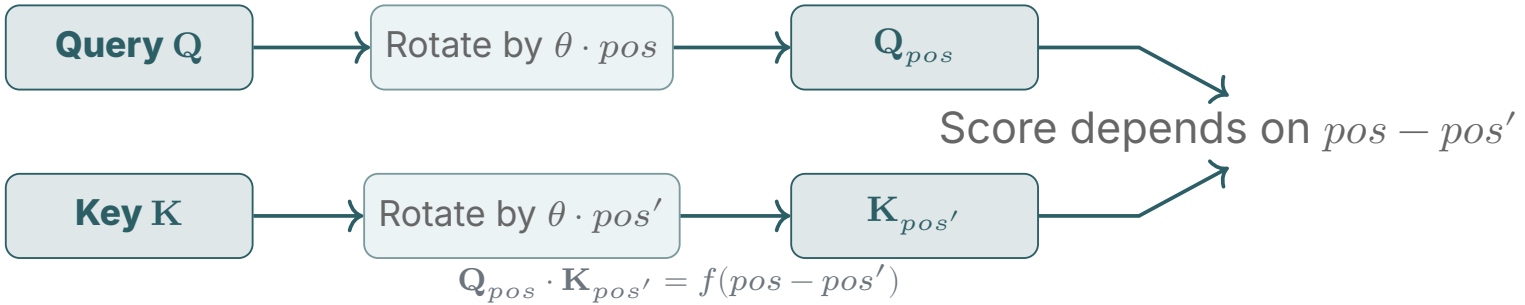
where $\mathbf{R}_{\Theta,pos}$ is a block-diagonal rotation matrix parameterised by position pos and frequency set Θ. The dot product $\mathbf{Q}_{pos} \cdot \mathbf{K}_{pos'}$ then depends only on the **relative offset** $pos - pos'$.

Rotary Positional Embedding --- RoPE

Why RoPE Became the Standard

- **Relative positions:** Attention score depends only on $pos - pos'$, not absolute values
- **No extra parameters:** Rotation is a fixed function of position
- **Extrapolation:** RoPE with frequency scaling enables 2-8$\times$ context extension beyond training length
- **Adopted by:** Llama 2, Llama 3, Mistral, Falcon, Qwen, Gemma

RoPE --- How Rotation Encodes Position



The attention score between two tokens depends only on their relative distance, not on where in the sequence they appear in absolute terms.

ALiBi --- Attention with Linear Biases

A Bias-Based Alternative to Positional Vectors

ALiBi (Press et al. 2022) does not modify embeddings at all. Instead, it subtracts a **linear penalty** from attention scores based on query-key distance:

$$\text{score}(i, j) = \mathbf{Q}_i \cdot \mathbf{K}_j - m \cdot |i - j|$$

where m is a head-specific slope (different per attention head). Nearby tokens get higher scores; distant tokens are penalised.

ALiBi --- Attention with Linear Biases

ALiBi Properties

- **No position vectors:** No extra parameters or embedding addition
- **Excellent extrapolation:** Models trained at 1K tokens can generalise to 2K–4K tokens with minimal quality loss
- **Adopted by:** MPT (MosaicML), BLOOM (BigScience)
- **Trade-off:** Enforces a recency bias that may not suit all tasks

Positional Encoding Schemes --- Comparison

Scheme	Learnable	Relative	Extrapolation	Used in
Sinusoidal (fixed)	No	Partial	Moderate	Original Transformer
Learned Absolute	Yes	No	Poor	GPT-2, BERT
RoPE	No	Yes	Good	Llama 2/3, Mistral, Gemma
ALiBi	No	Yes	Excellent	MPT, BLOOM
NoPE (no encoding)	N/A	Implicit	Limited	Some encoder models

How Positional Encoding Determines Context Window Size

The Link Between PE and Maximum Sequence Length

- **Learned absolute PE:** Context window is hard-limited to $L_{\max}$ set at training — extending requires fine-tuning
- **Sinusoidal:** Defined for any length, but quality degrades for positions far beyond training distribution
- **RoPE with YaRN / LongRoPE:** Frequency base θ can be scaled to extend Llama 3 from 8K to 128K tokens post-hoc
- **ALiBi:** Slope-based penalty allows clean extrapolation to 2–$4\times$ training length without retraining

How Positional Encoding Determines Context Window Size

Why This Matters for Practitioners

- The context window limit is not just a memory constraint — it is also a positional encoding design choice
- Extending context windows requires either retraining or applying positional interpolation/scaling techniques

GenAI Application 1 --- Long-Context Models

How Positional Encoding Enables 128K–1M Token Windows

- **Claude 3.5 (Anthropic):** 200K token context enabled by careful positional encoding design and efficient attention — the full context of a novel fits in one prompt
- **Gemini 1.5 Pro (Google):** 1M token window combines ring attention with positional schemes that scale to extreme lengths, enabling analysis of entire codebases and hour-long videos
- **Llama 3.1 (Meta):** Extended from 8K to 128K via LongRoPE frequency scaling applied after initial training — no architecture change required, only PE adjustment
- **GPT-4 Turbo (OpenAI):** 128K context built on top of learned positional embeddings fine-tuned on long documents

Every breakthrough in context window length traces back to improvements in positional encoding design and interpolation.

GenAI Application 2 --- RAG Systems and Position Awareness

Why Position Matters in Retrieval-Augmented Generation

- In a RAG prompt, the system instruction, retrieved passages, and user query are concatenated — each at a different position range
- **Lost-in-the-middle:** Research (Liu et al. 2023) shows LLMs with absolute PE attend disproportionately to early and late positions, missing critical information in the middle of long contexts
- **RoPE models** generalise better across positions because attention decays smoothly with relative distance
- **Prompt engineers** place the most important retrieved chunks at the beginning or end of the context to exploit PE-driven attention patterns
- Understanding PE explains why **chunk ordering** in RAG pipelines (LangChain, LlamaIndex) affects answer quality

Positional encoding is not just architecture theory — it directly shapes how practitioners structure production RAG prompts.

GenAI Application 3 --- Code Generation and Structured Data

Position Encoding in Code Models

- Code has strong positional structure: indentation level, line number, and scope boundaries all encode semantic meaning via position
- **CodeLlama (RoPE):** Relative position encoding allows the model to generalise to files longer than training examples
- **GitHub Copilot:** Uses context from the full open file; position of the cursor relative to function definitions drives completion quality
- **StarCoder2:** Trained with ALiBi-style biases on 80+ language corpora, enabling robust generalisation to long files

Structured Data Tables

For tabular data (CSV, SQL), column position in each row is encoded via the positional embedding — the model "knows" that column 3 consistently refers to the same field across rows, enabling accurate SQL generation.

GenAI Application 4 --- Positional Encoding Beyond Text

Positional Encoding in Vision and Multimodal Models

- **Vision Transformer (ViT):** Images are divided into 16×16 pixel patches; each patch receives a 2D learned positional embedding encoding its row and column on the image grid
- **DALL-E 3 / Stable Diffusion:** At each denoising step, spatial positional embeddings tell the model where on the canvas each feature is located — critical for coherent image composition
- **GPT-4V / LLaVA:** Vision patch tokens are inserted into the language sequence at fixed positions; 2D patch position is projected into the shared d_{model} embedding space
- **Audio (Whisper):** Spectrogram frames receive sinusoidal positional encodings so the model understands temporal order of speech

GenAI Application 5 --- Context Extension via PE Scaling

Practical Techniques for Extending Context Windows

- **Positional Interpolation (Chen et al. 2023):** Scale position indices down so a model trained at 4K tokens can be fine-tuned at 32K tokens with minimal compute
- **YaRN (Peng et al. 2023):** Modify RoPE frequency base θ from 10000 to 500000 to spread position information over longer ranges — used to extend Mistral and Llama to 32K–128K contexts
- **LongRoPE:** Applied by Meta to produce Llama 3.1 128K context from a base model trained at 8K; no full retraining required
- **Practical implication:** Choosing a base model pre-trained with RoPE gives practitioners more options for context extension than models using learned absolute embeddings

Common Misconceptions --- Part 1

Do Not Confuse These

- **“Positional encoding teaches the model what position means.”**
  No. It provides a positional signal; the model *learns* to use it during training. The encoding itself carries no inherent meaning until weights are trained to interpret it.
- **“Larger d_{model} always means better positional resolution.”**
  Not directly. The resolution of sinusoidal encoding depends on the frequency range, not simply on d_{model}.
- **“RoPE is added to the token embedding like sinusoidal PE.”**
  No. RoPE is applied *inside* the attention computation by rotating the Q and K vectors. The token embeddings themselves are unmodified.

Common Misconceptions --- Part 2

More Things to Watch Out For

- **“Transformers without positional encoding cannot process text.”**
  Partially false. They can process text as a bag-of-tokens — they just lose word-order information. Some encoder tasks do not require order (e.g., set classification).
- **“ALiBi is strictly better than RoPE.”**
  Context-dependent. ALiBi extrapolates better but enforces a hard recency bias. RoPE with YaRN achieves comparable extrapolation while allowing attention over arbitrary relative distances.
- **“Positional encoding only matters for long sequences.”**
  No. Even for 5-token sentences, word order is semantically critical (“dog bites man” vs “man bites dog”).

Summary

Positional Encoding — Key Takeaways

1. Self-attention is **permutation-equivariant**; it cannot distinguish token order without explicit positional information
2. **Sinusoidal PE** uses fixed sine/cosine patterns of increasing wavelength; bounded, unique, and allows relative-position computation
3. **Learned absolute PE** is trained end-to-end but cannot extrapolate beyond training sequence length
4. **RoPE** rotates Q/K vectors so attention scores encode only relative position — the standard in Llama, Mistral, and Gemma
5. **ALiBi** subtracts a linear distance bias from attention scores, enabling excellent length extrapolation without learned parameters
6. Positional encoding choice directly determines the model's **context window** limit and its ability to extend beyond it

What You Needed to Know Before This Session

Prerequisites in the Curriculum

- **Self-Attention Mechanism** — The $\mathbf{QK}^\top$ dot product and its permutation-equivariance
- **Multi-Head Attention** — Parallel attention heads and the shared input $\mathbf{X}$
- **Token Embeddings** — How tokens map to dense vectors of dimension d_{model}
- **Transformer Overview** — The high-level block structure

What This Concept Unlocks

Immediate Next Concepts

- Feed-Forward Networks in Transformers (Module 2.5)
- Decoder-Only Architecture (Module 2.6)
- Complete Transformer Block Walkthrough (Module 2.7)
- Tokenisation and Embeddings (Module 3)

Downstream Concepts

- Context Window Engineering (Module 5)
- Long-Context Fine-Tuning with YaRN / LongRoPE (Module 7)
- Efficient Inference and Context Compression (Module 9)
- Multimodal Positional Encoding in Vision Models (Module 6)

Next Steps

Recommended Actions

- Plot sinusoidal PE values for positions 1–100 using Python and matplotlib — observe the wavelength pattern
- Read: Vaswani et al. 2017 (Section 3.5, positional encoding)
- Read: Su et al. 2022 — “RoFormer: Enhanced Transformer with Rotary Position Embedding”
- Inspect a Llama 3 config file: find the `rope_theta` parameter value

Coming Up in Module 2

- Feed-Forward Networks — the second sublayer in each Transformer block
- Layer Normalisation and Residual Connections
- Decoder-Only Architecture — putting all components together
- Full Transformer Block walkthrough with parameter counts

Thank You

Positional Encoding | Module 2.2 | AI-TR-FN-TH-000004